```
> ZRANGE highscore -3 -1 WITHSCORES
1) "gamer2"
2) "50"
3) "gamer1"
4) "100"
5) "gamer3"
6) "150"
> quit
```

① Installs the Redis CLI

② Connects to the Redis cluster node, and replaces $CacheAddress with the output from the CloudFormation stack

③ Stores a session for user gamer1 with a time-to-live of 15 seconds

④ Gets the session for user gamer1

⑤ Wait 15 seconds and fetch the session for gamer1 again.

⑥ The response is empty, because the key was automatically deleted after 15 seconds.

⑦ Adds gamer1 with a score of 100 to the sorted set named highscore

⑧ Adds gamer2 with a score of 50 to the sorted set named highscore

⑨ Adds gamer3 with a score of 150 to the sorted set named highscore

⑩ Adds gamer4 with a score of 5 to the sorted set named highscore

Gets the top three gamers from the highscore list

The list is sorted ascending and includes the user followed by its score.

Quits the Redis CLI

You've successfully connected to the Redis cluster and simulated a session store and highscore list used by a typical gaming backend. With this knowledge, you could start to implement a caching layer in your own application. But as always, there are more options to discover.



Cleaning up

It's time to delete the CloudFormation stack named `redis-minimal`. Use the AWS Management Console or the following command to do so:

```
$ aws cloudformation delete-stack --stack-name redis-minimal
```

Continue with the next section to learn more about advanced deployment options with more than one node to achieve high availability or increase the available memory by using sharding.